



University of
Sheffield

COM1001 SPRING SEMESTER

Professor Phil McMinn

p.mcminn@sheffield.ac.uk

Test Coverage

Metrics for Testing

Test Coverage is a metric that describes **how much** of a system was tested by its tests (for example, unit and end to end tests). There are various ways to measure this.

Code Coverage is a type of **Test Coverage metric** that measures how much of our code was **executed** by the tests.

Line Coverage is a specific **Code Coverage metric** that measures how many code lines were executed.

Code Coverage tells us which parts of a system were **missed by our tests**, and what tests we may still need to write.

SimpleCov

SimpleCov is a gem that can track the lines of code executed by our RSpec tests.

We can set it up to work with our tests in `spec_helper.rb` as follows:

```
# Configure coverage logging
require "simplecov"
SimpleCov.start do
  add_filter "/spec/"
end
SimpleCov.coverage_dir "coverage"
```

This starts monitoring lines executed, ensuring we don't track the `spec/` directory – i.e., coverage of the tests themselves!

SimpleCov produces HTML reports in the `coverage/` directory of our app

SimpleCov Monitors Our Tests

We can supply `rspec` command with a directory name and it will run all the tests in it – anything ending in `_spec.rb`

```
codio@north-mister:~/workspace/com1001-code/forms/football_players$ rspec spec/features/
```

```
.....
```

```
Finished in 2.4 seconds (files took 2.38 seconds to load)
```

```
12 examples, 0 failures
```

```
Coverage report generated for RSpec to /home/codio/workspace/com1001-code/forms/football_players/coverage. 91 / 92 LOC (98.91%) covered.
```

```
codio@north-mister:~/workspace/com1001-code/forms/football_players$
```

This line is output by SimpleCov.

SimpleCov only counts code in `.rb` files, i.e., our controllers and models etc. It does not monitor coverage in `.erb` files. Hence only 92 **lines of code (LOC)** in total, and not more.

It says we executed (covered) 91/92 LOC.

We can find out which line the feature tests missed by checking out the coverage report.

Codio - com1001

Code coverage for Football play

https://codio.com/pmcminn/com1001/preview/com1001-code%2Fforms%2Ffootball_players%2Fcoverage%2Findex.html150%

CodioProjectFileEditFindViewToolsEducationHelpRunProject Index (static)Configure....PMCMINN

Filetree

PMCMINN
com1001

com1001 (master)

com1001-code

basics

controllers

cookies

databases

debugging

forms

football_players

controllers

coverage

assets

.last_run.json

.resultset.json

.resultset.json.lock

index.html

db

helpers

models

public

spec

views

.rubocop.yml

app.rb

Gemfile

Gemfile.lock

Terminalindex.html - ...

https://north-mister.codio.io/com1001-code/forms/football_players/coverage/index.html

All Files (98.91%)Generated less than a minute ago

All Files (98.91% covered at 8.16 hits/line)

8 files in total.
92 relevant lines, 91 lines covered and 1 lines missed. (98.91%)

Search:

File	% covered	Lines	Relevant Lines	Lines covered	Lines missed	Avg. Hits / Line
controllers/delete.rb	85.71 %	11	7	6	1	0.86
app.rb	100.00 %	17	8	8	0	1.00
controllers/add.rb	100.00 %	16	10	10	0	6.60
controllers/edit.rb	100.00 %	21	13	13	0	2.31
controllers/search.rb	100.00 %	15	6	6	0	11.50
db/db.rb	100.00 %	11	7	7	0	1.00
helpers/validation.rb	100.00 %	18	10	10	0	23.10
models/player.rb	100.00 %	47	31	31	0	10.77

Showing 1 to 8 of 8 entries

Generated by simplecov v0.21.2 and simplecov-html v0.12.3 using RSpec

To open the report in Codio, go to the **coverage** directory in the file tree. Right click on **index.html** and click **“Preview static”**

Ideally, we want the **coverage percentage** to be as high as possible, and **hits/line** to be equal or greater to 1.

A “**hit**” is an execution of a line. “**hits/line**” is the number of times, on average, each line in the system was executed by the tests.
The more hits a line has, the more it has been executed. So we can see which files have been executed the most.

All Files (98.91%)Generated about a minute ago

All Files (98.91% covered at 8.16 hits/line)

8 files in total.
92 relevant lines, 91 lines covered and 1 lines missed. (98.91%)

Search:

File	% covered	Lines	Relevant Lines	Lines covered	Lines missed	Avg. Hits / Line
 controllers/delete.rb	85.71 %	11	7	6	1	0.86
 app.rb	100.00 %	17	8	8	0	1.00
 controllers/add.rb	100.00 %	16	10	10	0	6.60
 controllers/edit.rb	100.00 %	21	13	13	0	2.31
 controllers/search.rb	100.00 %	15	6	6	0	11.50
 db/db.rb	100.00 %	11	7	7	0	1.00
 helpers/validation.rb	100.00 %	18	10	10	0	23.10
 models/player.rb	100.00 %	47	31	31	0	10.77

Showing 1 to 8 of 8 entries

Coverage Tracking

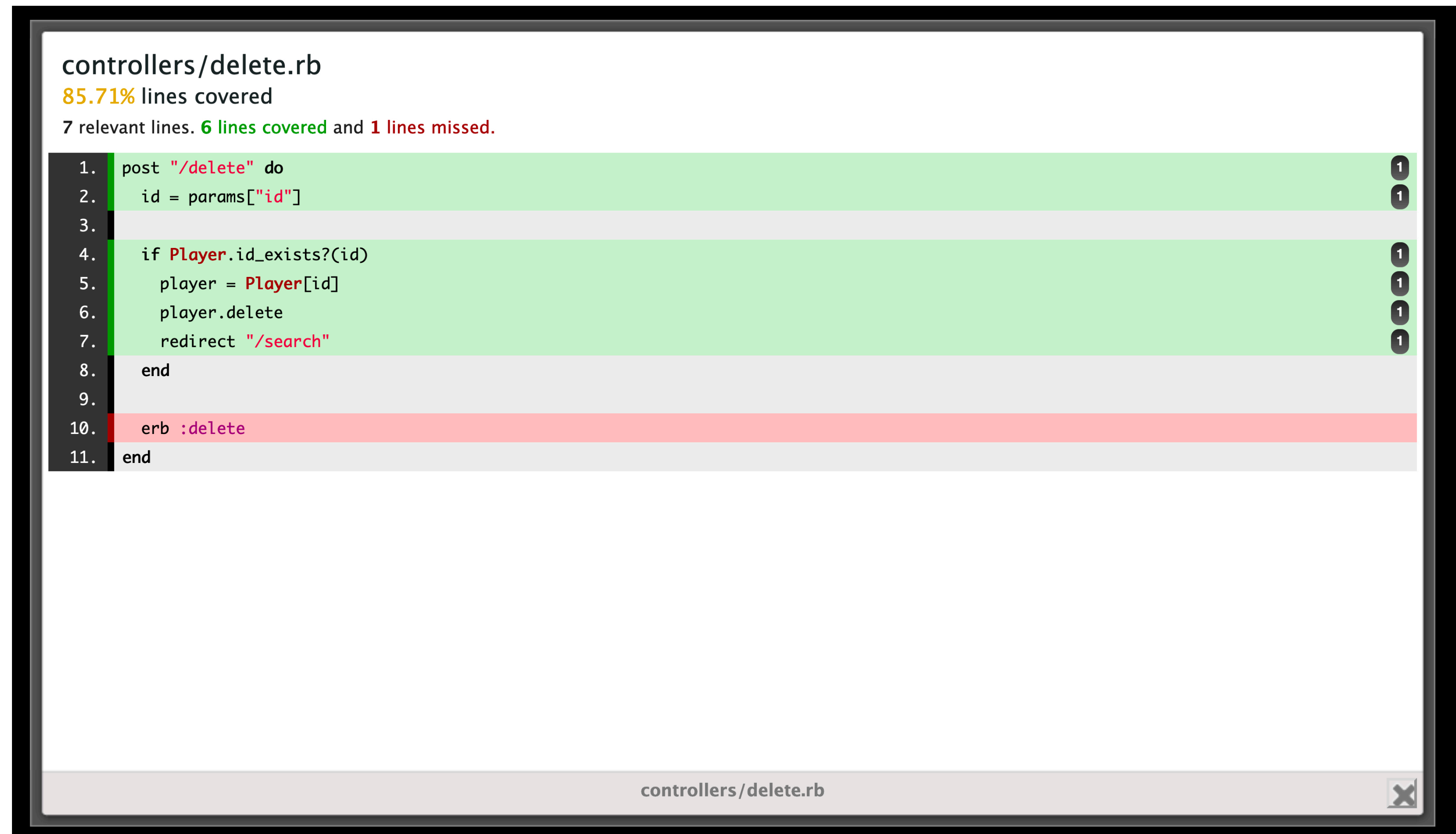
We can see which lines were covered and not covered in each individual file.

Covered lines are in green,
uncovered lines are in red.

Whitespace/negligible code is in grey.

None of our feature tests executed the condition on line 4 as false, hence they never reached line 10, which displays an error message (player does not exist).

Can you think why this is not possible?



```
controllers/delete.rb
85.71% lines covered
7 relevant lines. 6 lines covered and 1 lines missed.

1. post "/delete" do
2.   id = params["id"]
3.
4.   if Player.id_exists?(id)
5.     player = Player[id]
6.     player.delete
7.     redirect "/search"
8.   end
9.
10.  erb :delete
11. end
```


controllers/delete.rb

85.71% lines covered

7 relevant lines. 6 lines covered and 1 lines missed.

```
1. post "/delete" do
2.   id = params["id"]
3.
4.   if Player.id_exists?(id)
5.     player = Player[id]
6.     player.delete
7.     redirect "/search"
8.   end
9.
10.  erb :delete
11. end
```

controllers/delete.rb

A user of our system can only delete players via the edit page, and they can only edit players already added to the database.

So there is no way for them to delete a non-existent player.

Remember we cannot visit POST routes in our browser, so nor can Capybara.

So we can never cover this line via an end to end test. However, we need to test what happens if we did try to delete a player that did not exist (e.g., if attempted by a robot)

We can still do this via a unit test...

Additional Unit Test

```
describe "POST/delete" do
  it "does not allow deletion of a invalid player" do
    post "/delete", "id" => "-100"
    expect(last_response.body).to include("Unknown player")
  end
end
```

forms/football_players/spec/unit/delete_spec.rb

This is why we need to have both **end-to-end tests** and **unit tests**:

Unit tests ensures the app works beyond what the user can see, by testing edge cases...

... but unit tests cannot, on their own, guarantee all the units will work together.

If we don't supply any parameters to the **rspec** command it will run all of our tests, feature and unit – anything ending in **_spec.rb**

```
codio@north-mister:~/workspace/com1001-code/forms/football_players$ rspec
```

```
.....
```

```
Finished in 2.38 seconds (files took 2.69 seconds to load)
```

```
15 examples, 0 failures
```

```
Coverage report generated for RSpec to /home/codio/workspace/com1001-code/forms/football_players/coverage. 92 / 92 LOC (100.0%) covered.
```

```
codio@north-mister:~/workspace/com1001-code/forms/football_players$
```

controllers/delete.rb

100.0% lines covered

7 relevant lines. 7 lines covered and 0 lines missed.

1.	post "/delete" do	1
2.	id = params["id"]	2
3.		
4.	if Player .id_exists?(id)	2
5.	player = Player [id]	1
6.	player.delete	1
7.	redirect "/search"	1
8.	end	
9.		
10.	erb :delete	1
11.	end	

controllers/delete.rb

All lines of code now covered

Code Coverage: Caveats

Code Coverage is primarily a measure of **test effort**, *not* **test effectiveness**

Merely executing a line of code in a test does not guarantee that it is bug free.

High coverage increases our confidence in correctly working software, but writing tests *just* to achieve coverage is not always sensible or a good use of time.